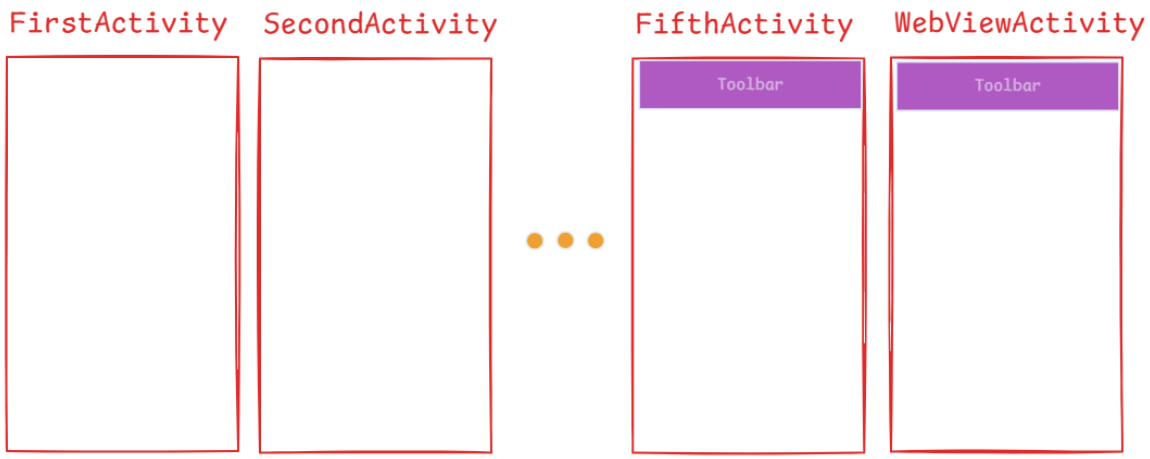
PERTEMUAN 7 - mus

▼ 📂 Activity dan Fragment

Kenapa dengan Activity?

Selama ini, setiap kali kita membuat halaman baru, kita biasanya membuat **Activity** baru dan menambahkan komponennya hingga membentuk tampilan yang diinginkan.



Namun, pendekatan ini memiliki beberapa kekurangan:

- Tiap Activity memiliki **UI sendiri-sendiri**, sehingga saat berpindah antar Activity, Android harus memuat ulang seluruh tampilan.
- Akibatnya, pada aplikasi dengan banyak layar (misalnya 5 Activity), perpindahan jadi lebih lambat dan bisa memicu *lag* pada perangkat berspesifikasi rendah

Selain itu, Activity juga memiliki keterbatasan dalam hal **navigasi dan UI**:

- Navigasi antar Activity **lebih lambat** dibanding Fragment karena Activity harus dimuat ulang.
- Tidak bisa menampilkan elemen UI yang tetap di semua layar (seperti sidebar atau bottom navigation).

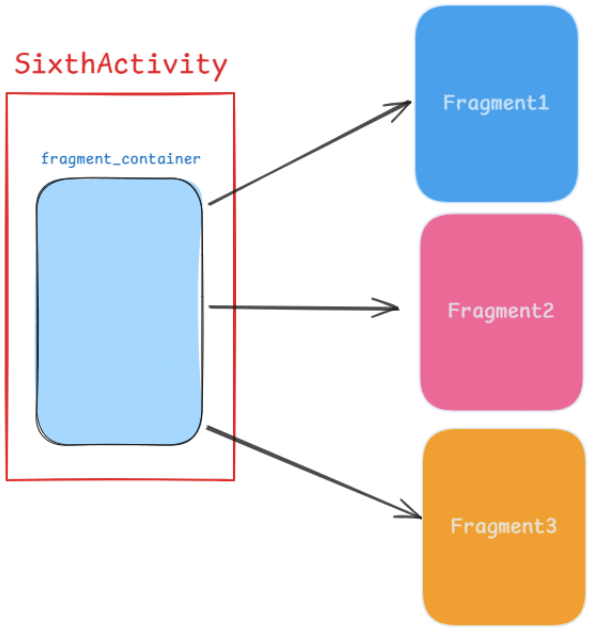
> Contoh :
>
> *Aplikasi seperti YouTube atau Instagram menggunakan **Fragment**, sehingga bagian **Bottom Navigation** tetap ada saat berpindah antar layar.*

⚠️ Activity cocok digunakan saat:

- Aplikasi hanya memiliki 2–3 halaman, atau
- Tiap halaman memiliki fungsi yang benar-benar berbeda, seperti `LoginActivity` dan `HomeActivity`.

Mengapa Fragment?

Untuk mengatasi kekurangan Activity, Android menyediakan **Fragment**, yaitu bagian dari UI yang dapat digunakan kembali di dalam satu Activity.



Dengan Fragment:

- Mengurangi jumlah Activity, sehingga aplikasi lebih ringan dan cepat
- Navigasi menjadi lebih fleksibel (menggunakan Bottom Navigation, ViewPager, atau TabLayout) untuk menampilkan halaman berbeda.
- Fragment bisa dimuat atau dilepas sesuai kebutuhan tanpa membuat ulang Activity.

▼ 📂 Penggunaan Fragment

📌

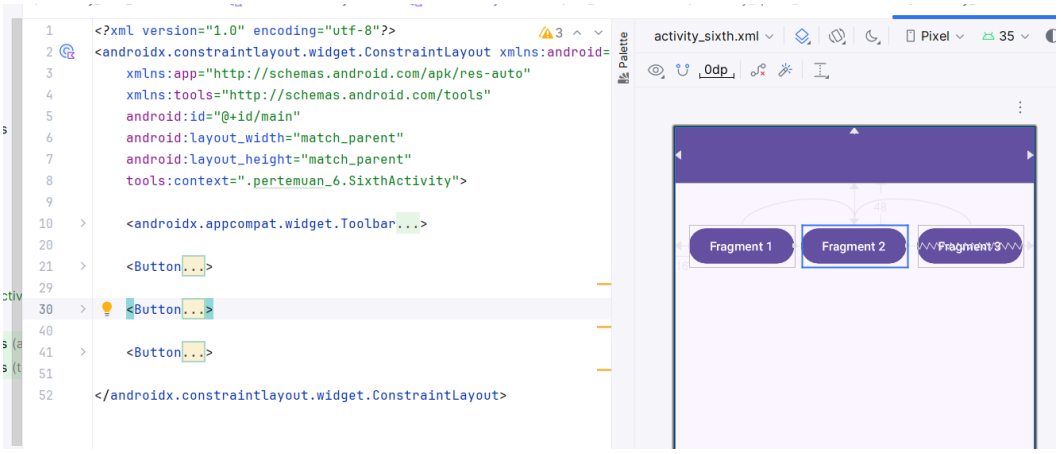
- Buat Activity `SeventhActivity` di `pertemuan_7`
- Terapkan `Toolbar` beserta tombol back
- Tambah tombol “Pertemuan 6” di `MainActivity` yang mengarah ke `SeventhActivity`

Kita akan coba terapkan 3 Fragment berbeda pada `SixthActivity`

▼

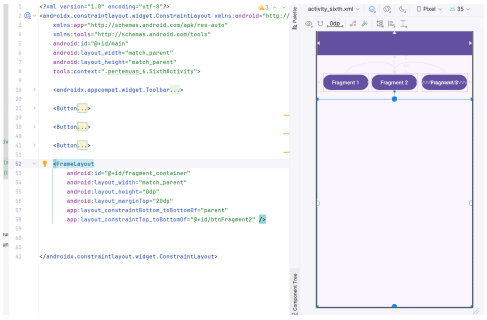
1️⃣ Buat sebuah container dan button untuk berpindah frament

- Pada `activity_sixth.xml` buatlah 3 buah `Button` dengan id `btnFragment1`, `btnFragment2` , `btnFragment3` tepat dibawah Toolbar



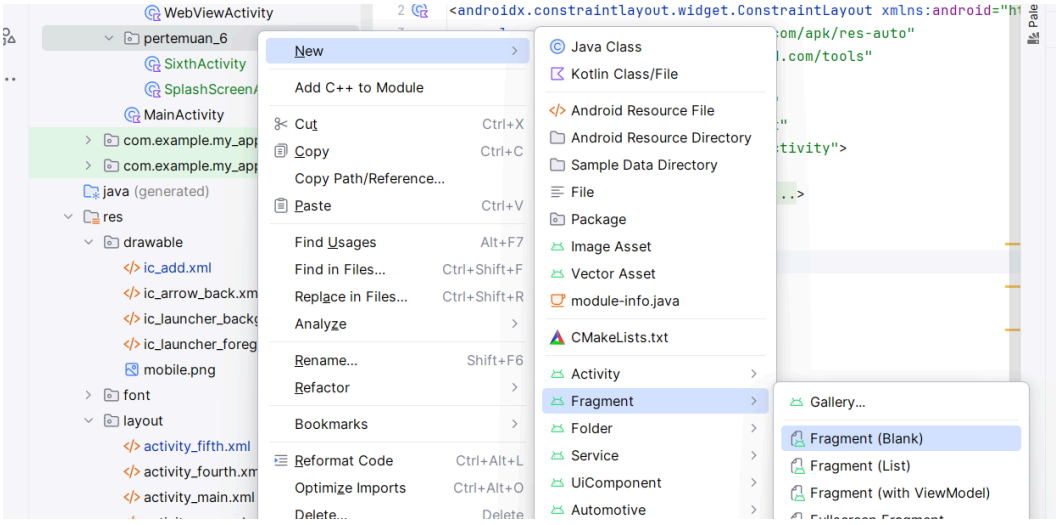
- Tambahkan sebuah `FrameLayout` sebagai container fragment, pastikan constraint terpasang dan `layout_hegiht = 0dp`

```
    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_marginTop="20dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/btnFragment2" />
```

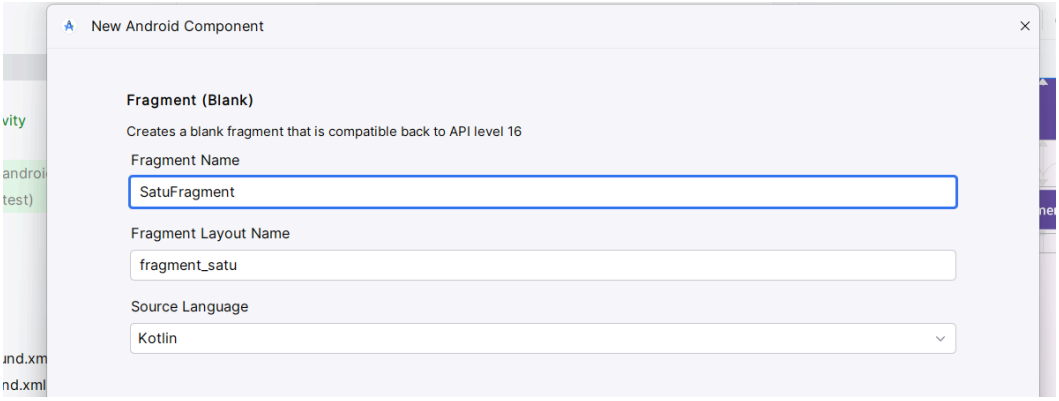


▼ 2️⃣ Buat Fragment yang dibutuhkan

- Klik kanan di `pertemuan_6` → New → Fragment → Fragment (Blank)



- Buat Fragment baru dengan nama `SatuFragment` lalu klik finish



- Silahkan buka `SatuFragment` , hilangkan beberapa baris kode yang tidak diperlukan. Cukup tinggalkan `onCreateView()`

Before

```
package com.example.my_apps.pertemuan_6

import ...

// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"

/**
 * A simple [Fragment] subclass.
 * Use the [SatuFragment.newInstance] factory method to
 * create an instance of this fragment.
 */
class SatuFragment : Fragment() {
    // TODO: Rename and change types of parameters
    private var param1: String? = null
    private var param2: String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        arguments?.let {
            param1 = it.getString(ARG_PARAM1)
            param2 = it.getString(ARG_PARAM2)
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_satu, container, attachToRoot: false)
    }

    companion object {
        /**
         * Use this factory method to create a new instance of
         * this fragment using the provided parameters.
         *
         * @param param1 Parameter 1.
         * @param param2 Parameter 2.
```

After

```
package com.example.my_apps.pertemuan_6

import ...

class SatuFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_satu, container, attachToRoot: false)
    }
}
```

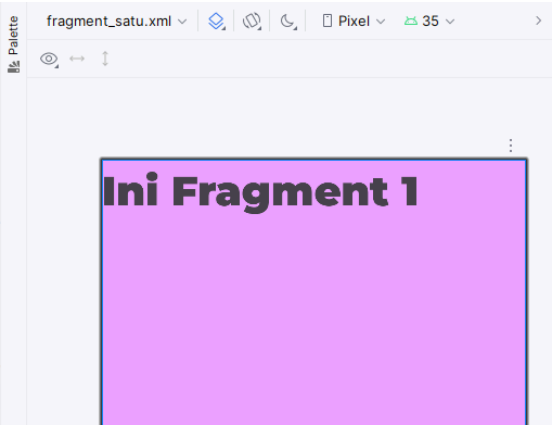
Cukup tinggalkan `onCreateView()`

- Buka `fragment_satu.xml` dan ubah text serta bakground sesuai yang diinginkan

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.androi
    xmlns:tools="http://schemas.android.com/tools
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#EFA2FF"
    tools:context=".pertemuan_6.SatuFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fontFamily="@font/monsterrat_blac
        android:textSize="40sp"
        android:text="Ini Fragment 1" />

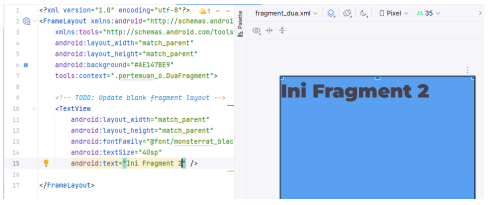
</FrameLayout>
```

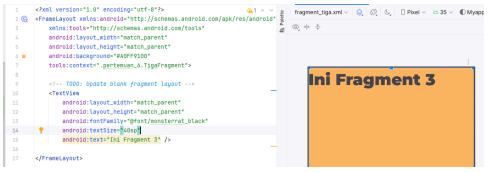


- Buat `DuaFragment` dan `TigaFragment` , sederhanakan kode-nya serta perbaharui layout-nya









▼ 3️⃣ Definisikan `fragement_container` di `SixthActivity`

- Tambahkan kode berikut dibawah `onCreate()`

```
override fun onCreate(savedInstanceState: Bundle?)
{
    ...
}

private fun replaceFragment(fragment: Fragment) {
    supportFragmentManager.beginTransaction()
        .replace(binding.fragmentContainer.id, frag
ment)
        .addToBackStack(null)
        .commit()
}
```

- Panggil function `replaceFragment` di `SixthActivity`

```
    override fun onCreate(savedInstanceState: Bundl
e?) {
        ...

        // Menampilkan fragment pertama secara defa
```

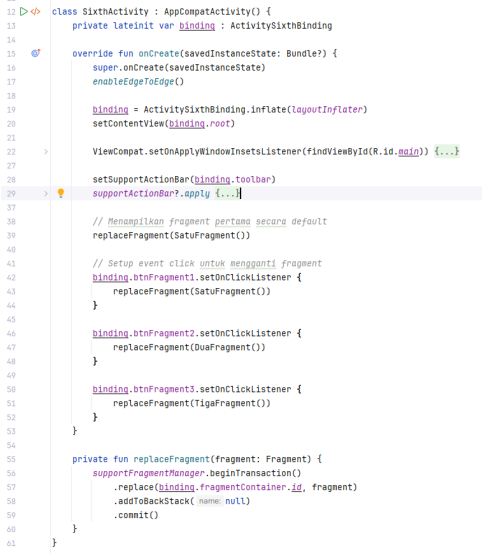
```
ult
        replaceFragment(SatuFragment())

        // Setup event click untuk mengganti fragme
nt
        binding.btnFragment1.setOnClickListener {
            replaceFragment(SatuFragment())
        }

        binding.btnFragment2.setOnClickListener {
            replaceFragment(DuaFragment())
        }

        binding.btnFragment3.setOnClickListener {
            replaceFragment(TigaFragment())
        }
    }
```

- Hasil akhir `SixthActivity()` setelah dimodifikasi, silahkan coba dijalankan.



```
class SixthActivity : AppCompatActivity() {
    private lateinit var binding : ActivitySixthBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        binding = ActivitySixthBinding.inflate(layoutInflater)
        setContentView(binding.root)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) {...}

        setSupportActionBar(binding.toolbar)
        supportActionBar?.apply {...}

        // Menampilkan fragment pertama secara default
        replaceFragment(SatuFragment())

        // Setup event click untuk mengganti fragment
        binding.btnFragment1.setOnClickListener {
            replaceFragment(SatuFragment())
        }

        binding.btnFragment2.setOnClickListener {
            replaceFragment(DuaFragment())
        }

        binding.btnFragment3.setOnClickListener {
            replaceFragment(TigaFragment())
        }
    }

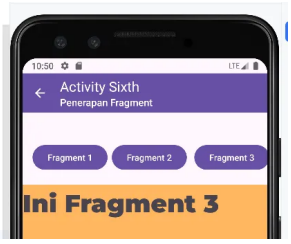
    private fun replaceFragment(fragment: Fragment) {
        supportFragmentManager.beginTransaction()
            .replace(binding.fragmentContainer.id, fragment)
            .addToBackStack( name: null)
            .commit()
    }
}
```

- `SixthActivity` menampilkan `SatuFragment()` secara default ketika Activity dimulai.
- Tombol-tombol disiapkan untuk mengganti fragment saat ditekan.
- `.replace(binding.fragmentContai fragment)` digunakan untuk mengganti fragment di dalam `fragment_container`.
- `addToBackStack(null)` ditambahkan untuk memungkinkan pengguna kembali ke fragment

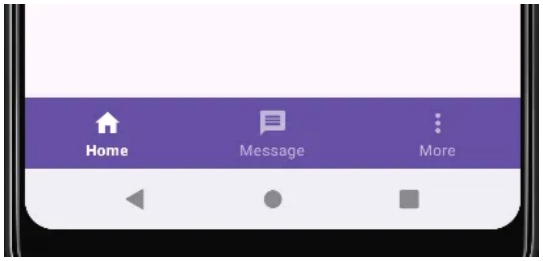
sebelumnya dengan tombol back.







▼ 📂 Bottom Navigation

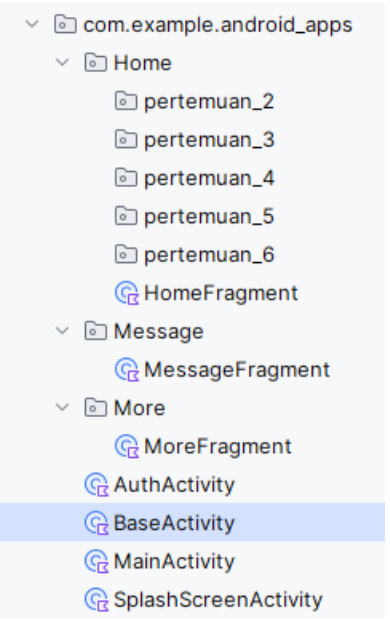


Kita akan buat `BaseActivity` dan menerapkan Bottom Navigation dengan 3 screen yaitu **Home**, **Message** dan **More.**

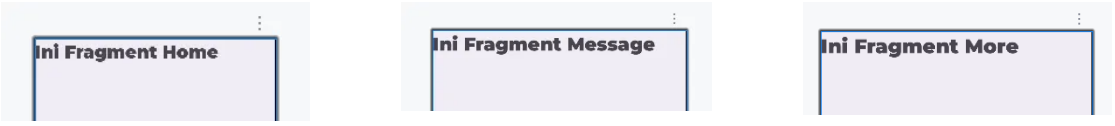
▼ 1️⃣ Directory & Fragment

- Buat `BaseActivity` sejajar dengan `MainActivity`
- Ubah `AndroidManifest` untuk memanggil `BaseActivity` saat pertama kali *(by pass halaman login sementara)*
- Buat package `Home` , `Message` dan `More` dan masing-masing package buat `HomeFragment` , `MessageFragment` dan `MoreFragment`
- Pindahkan semua package pertemuan 2-6 ke `Home`

Pastikan sama struktur folder-nya!



Cukup desain layout fragment dengan text sederhana dahulu saja:



▼ 2️⃣ FrameLayout & BottomNavigationView

Buat resource file di `/res/menu` dengan nama `bottom_nav_menu.xml`:

▼ bottom_nav_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/home"
        android:icon="@drawable/ic_home"
        android:title="Home" />

    <item
        android:id="@+id/message"
        android:icon="@drawable/ic_message"
        android:title="Message" />

    <item
        android:id="@+id/more"
        android:icon="@drawable/ic_more"
        android:title="More" />

</menu>
```

📌 *Silahkan cari vector asset untuk `ic_home` , `ic_message` dan `ic_more`*

Pada `activity_base.xml`, terapkan `FrameLayout` dan `BottomNavigationView`:

▼ **activity_base.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/a
ndroid"
    xmlns:app="http://schemas.android.com/apk/res-a
uto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".BaseActivity">

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        app:layout_constraintBottom_toTopOf="@+id/b
ottom_nav_view"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent" /
>

    <com.google.android.material.bottomnavigation.B
ottomNavigationView
        android:id="@+id/bottom_nav_view"
        style="@style/Widget.MaterialComponents.Bot
tomNavigationView.Colored"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="par
ent"
        app:menu="@menu/bottom_nav_menu" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout
>
```

Pada `BaseActivity` , inisialisasi `bottomNavView` dan terapkan `setOnItemSelectedListener`

```
override fun onCreate(savedInstanceState: Bundle?) {
        private lateinit var binding : ActivityBaseBin
ding

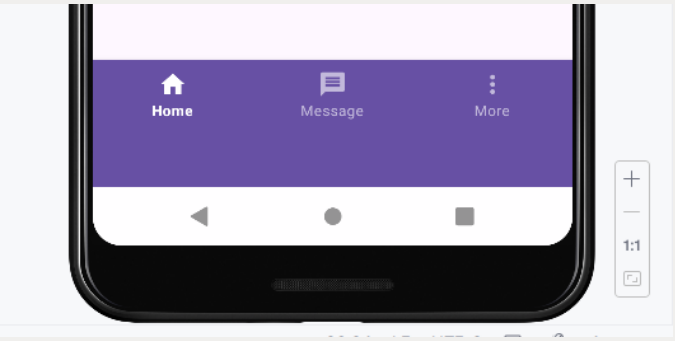
    ...

    binding.bottomNavView.setOnItemSelectedListener {
        when (it.itemId) {
            R.id.home -> {
                Toast.makeText(this, "Home Clicked", T
oast.LENGTH_SHORT).show()
                true
            }
            R.id.message -> {
                Toast.makeText(this, "Message Clicke
d", Toast.LENGTH_SHORT).show()
                true
            }
            R.id.more -> {
                Toast.makeText(this, "More Clicked", T
oast.LENGTH_SHORT).show()
                true
            }
            else -> false // return false jika item ti
dak ada yang di klik
        }
    }
```

```
}
```

Jalankan aplikasi untuk melihat apakah BottomNavigation sudah tampil dan setiap icon bisa di-klik.

⚠️ Jika Navigasi memilki jarak di bagian bawah seperti berikut:

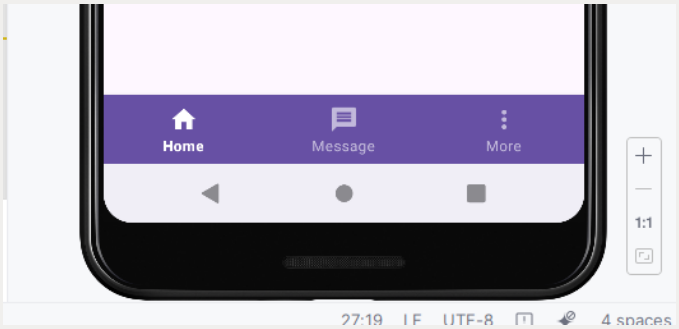


Kita perlu ubah `systemBars.bottom` menjadi 0 di `BaseActivity():`
Before:

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
    insets
}
```

After:

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, bottom: 0)
    insets
}
```



▼ 3️⃣ Replace Fragment

Pada `BaseActivity` , tambahkan function `replaceFragment()`

```
override fun onCreate(...) { ... }


private fun replaceFragment(fragment: Fragment) {
    supportFragmentManager.beginTransaction()
        .replace(binding.fragmentContainer.id, fragmen
t)
        //.addToBackStack(null) -> ini kita nonaktifka
n agar saat back langsung keluar aplikasi
        .commit()
}
```

Pada `onCreate()`, sesuaikan `bottomNavView` :

```
override fun onCreate(savedInstanceState: Bundle?) {
      ...

      /** FragmentHome sebagai fragment default */
      replaceFragment(HomeFragment())

      binding.bottomNavView.setOnItemSelectedListener
{
```

```
            when (it.itemId) {
                R.id.home -> {
                    replaceFragment(HomeFragment())
                    true
                }
                R.id.message -> {
                    replaceFragment(MessageFragment())
                    true
                }
                R.id.more -> {
                    replaceFragment(MoreFragment())
                    true
                }
                else ->false
            }
        }
    }
```

Jalankan aplikasi dan pastikan BottomNavigation berhasil diterapkan

▼ 4️⃣ Penyesuaian Layout fragment_home.xml

Sekarang kita akan mengisi `fragment_home` dengan tombol Pertemuan 2-6 dan komponen lain yang ada di `activity_main.xml`

Before

```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#11364A5F"
    tools:context=".FragmentHome">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fontFamily="@font/monsterrat_black"
        android:textSize="30sp"
        android:text="Ini Fragment Home" />

</FrameLayout>
```

After

```xml
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    tools:context=".Home.HomeFragment">

    <TextView...>

    <TextView...>

    <Button...>

    <Button...>

    <Button...>

    <Button...>

    <Button...>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Ubah

- `FrameLayout` → `ConstrainLayout`

- `TextView` "Ini Fragment Home" → Text Welcome dan List Button Pertemuan 2-6

▼ 5️⃣ Penyesuaian HomeFragment

Sekarang kita akan menyesuaikan `HomeFragment` dengan kode binding yang ada di `MainActivity` agar setiap tombol berfungsi kembali

- Definisikan binding di fragment

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = nu
ll
    private val binding get() = _binding!!

    override fun onCreateView(...){
        /** Hapus yang lama */
        //return inflater.inflate(R.layout.fragment
_message, container, false)

        /** Ganti menjadi versi binding */
        _binding = FragmentHomeBinding.inflate(infl
ater, container, false)
            return binding.root
    }

}
```

Pada Fragment, binding di-definisikan pada onCreateVIew

- Buat function baru `onViewCreated()` sejajar
  (cukup ketikkan saja dibawah dan pilih dari autocomplete yang tersedia)



lalu salin pemanggilan binding tombol pertemuan 2-6 maupun Logout kedalam nya

```
    override fun onCreateView(...): View? {
        ...
    }

    override fun onViewCreated(view: View, savedIns
tanceState: Bundle?) {

        binding.btnToSecond.setOnClickListener{...}

        binding.btnToThird.setOnClickListener{...}

        binding.btnToFourth.setOnClickListener{...}

        binding.btnToFifth.setOnClickListener{...}

        binding.btnToSixth.setOnClickListener{...}

        /** binding lainnya */
    }
```

Sesuaikan dengan project masing-masing

untuk mengatasi error pada `Intent` , sesuaikan `this` → `requireContext()`

Before

```
binding.btnToSecond.setOnClickListener{
    val intent = Intent(this, SecondActivity::class.java)
    intent.putExtra("judul","Pertemuan 2")
    intent.putExtra("description","Deskripsi materi yang dibahas saat Pertemuan 2")
```

After

```
binding.btnToSecond.setOnClickListener{
    val intent = Intent(requireContext(), SecondActivity::class.java)
    intent.putExtra( name: "judul", value: "Pertemuan 2")
```

⚠️ Jika ada error pada `sharedPref` dan `finish()`, silahkan sesuaikan pemanggilannya seperti berikut:

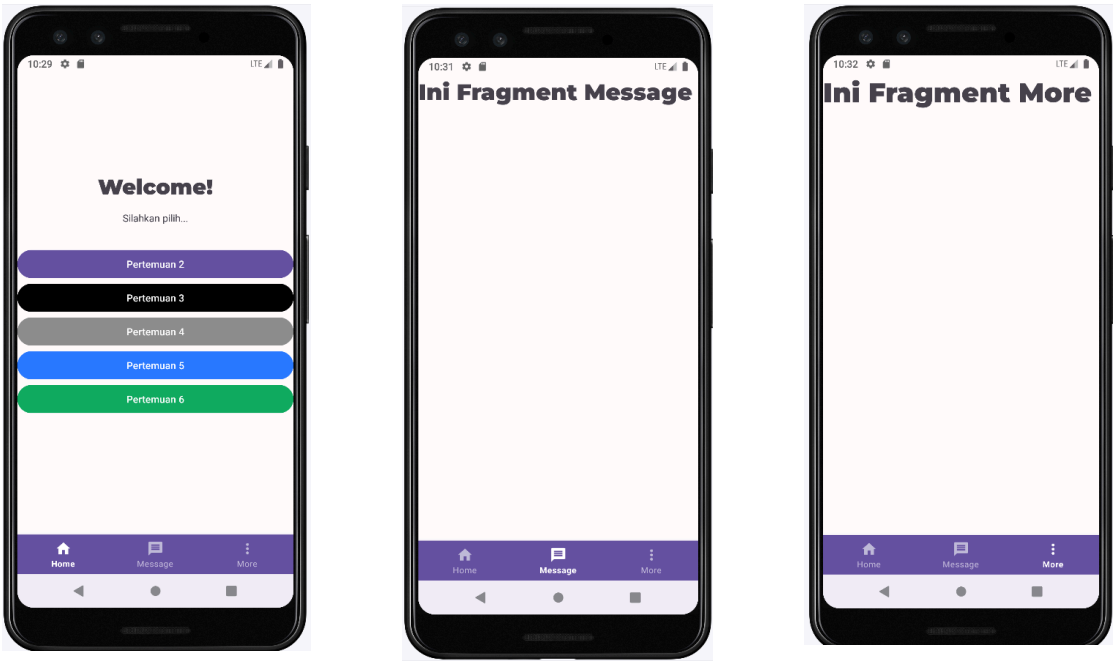
```
val sharedPref = requireContext().getSharedPreferences(...)
```

penambahan requireContext() karena sedang berada di Fragment

```
requireActivity().finish()
```

Karena posisi kita sedang ada di fragment, saat logout kita harus destroy BaseActivity-nya dengan menambahkan requireActivity() saat memangil finish()

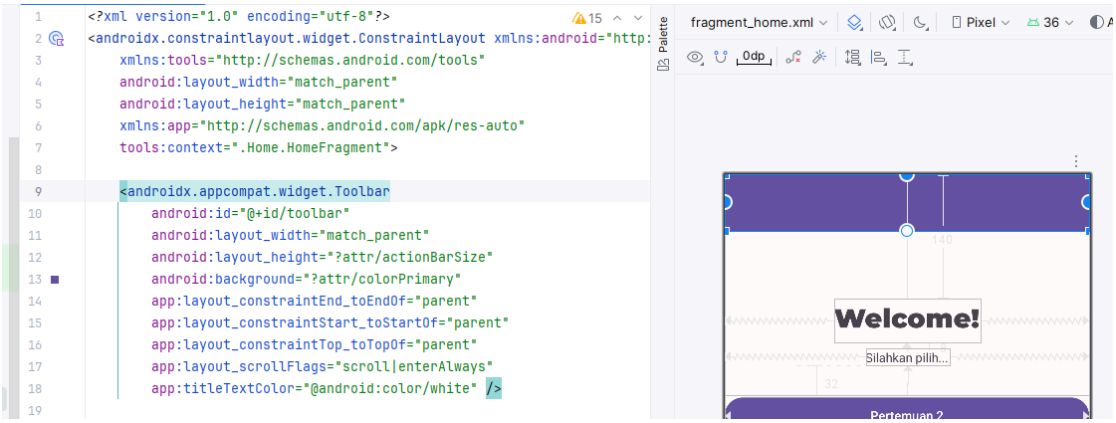
Hasil akhir:



▼ 6️⃣ Terapkan Toolbar

Terapkan `Toolbar` di setiap fragment
(*tidak perlu coordinatorLayout, cukup tambah Toolbar dan set constraint seperti biasa)*

Contoh pada `fragment_home.xml`



Pada `HomeFragment`, tambahkan kode berikut pada `onViewCreated()`

```
override fun onViewCreated(view: View, savedInstanceSt
ate: Bundle?) {

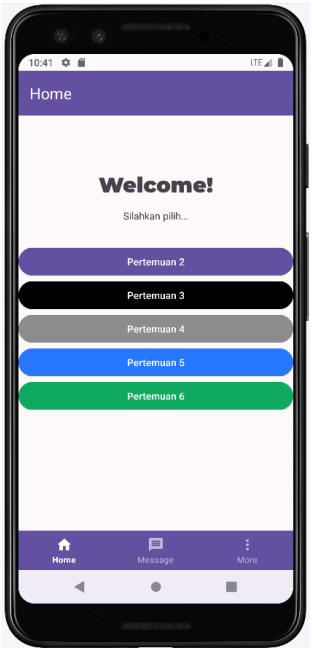
        (requireActivity() as AppCompatActivity).setSu
pportActionBar(binding.toolbar)
        (requireActivity() as AppCompatActivity).suppo
rtActionBar?.apply {
            title = "Home"
        }

        /** Kode binding tombol lainnya */
        ...
}
```

berbeda dengan Activity, Fragment harus menggunakan `(requireActivity() as AppCompatActivity)` saat menggunakan Toolbar

Terapkan untuk `MessageFragment` dan `MoreFragment` *(jangan lupa ubah FrameLayout menjadi ConstraintLayout)*

hasil akhir setelah Toolbar terpasang:







📌 Pada tahap ini `BaseActivity` yang akan menjadi activity utama, sehingga:

- `MainActivity` sudah tidak digunakan lagi, silahkan dihapus
- Pastikan alur `SplasScreenActivity` dan `AuthActivity` diarahkan ke `BaseActivity`
